StoryStream — A Social Media Platform

Somichui Zimik
Dept. of Computer Science & Engineering
Roll No: 439

Sonal Verma
Dept. of Computer Science & Engineering
Roll No: 440

Stanzin Kunsal
Dept. of Computer Science & Engineering
Roll No: 443

***Abstract*—StoryStream is a high-performance Relational Database Management System (RDBMS) architected to manage the complex, high-concurrency data requirements inherent in modern social media platforms. This project explores the transition from conceptual requirements to a fully realized production-ready schema, utilizing MySQL 8.0 as the core engine. The system is designed with a strict adherence to 3NF/BCNF normalization principles to minimize data redundancy while maintaining relational integrity across nine primary entities, including users, posts, and self-referencing follow structures.**

The implementation leverages a full-stack architecture consisting of a Node.js and Express.js backend communicating via a RESTful JSON API. Critical database engineering concepts are integrated to ensure system reliability: ACID-compliant transactions are utilized to manage multi-table operations like post creation and hashtag linking, while pessimistic locking and specific isolation levels are employed to mitigate concurrency issues such as "lost updates" in high-traffic scenarios. Furthermore, the system implements "defense-in-depth" security through custom triggers for automated audit logging and check constraints for server-side data validation. This report provides a detailed analysis of the system architecture, ER modeling, implementation of advanced database logic, and the security protocols established to ensure a scalable and robust data layer for social interaction.

***Index Terms*—Introduction, Table Descriptions, Relationships, ER Diagram, Implementation, Transaction and Concurrency Control, Assertion and Triggers, Security, Result, Conclusion, Future Scope**

I. Introduction

A. System Overview

StoryStream is a full-stack, Twitter-like social media platform that enables users to create short-form posts (limited to 280 characters), follow other users, like and comment on content, discover posts through hashtags, and receive real-time in-app notifications. The system is designed with a clear separation between the frontend (React 18 + Vite) and the backend (Node.js + Express.js + MySQL), communicating exclusively through a RESTful JSON API.

B. Core Features

Summary of the functional requirements and core capabilities integrated into the StoryStream environment:

TABLE I
StoryStream Core Features

Feature	Description
User Authentication	Registration, login, and JWT-based session management
Post Management	Create, read, and soft-delete posts (max 280 characters)
Follow System	Directed follower/following relationships between users
Like System	Idempotent likes with animated feedback on the frontend
Comment System	Threaded inline comments on posts
Hashtag Discovery	Automatic extraction, normalization, and searchable hashtag pages
Notifications	Real-time in-app alerts for likes, comments, and follows
User Search	Search by username or bio with paginated results
Dark/Light Mode	Animated theme toggle with persistence
Audit Trail	Immutable audit_logs table recording all significant actions

II. Table Descriptions

A. Table 1: users

Stores user information and acts as the central entity referenced by multiple tables.

Key Attributes:

- id (Primary Key)
- username, email (Unique)
- password (Encrypted)
- created_at

B. Table 2: posts

Stores user-generated posts and maintains a relationship with the users table.

Key Attributes:

- id (Primary Key)
- user_id (Foreign Key → users)
- content
- created_at

C. Table 3: comments

Stores comments made by users on posts.

Key Attributes:

- id (Primary Key)
- post_id (Foreign Key → posts)
- user_id (Foreign Key → users)
- content

D. Table 4: likes

Represents user likes on posts, ensuring each user can like a post only once.

Key Attributes:

- id (Primary Key)
- user_id (Foreign Key → users)
- post_id (Foreign Key → posts)

E. Table 5: follows

Models the relationship between users in a follower–following structure.

Key Attributes:

- follower_id (Foreign Key → users)
- following_id (Foreign Key → users)

Note: This is a self-referencing relationship.

F. Table 6: hashtags

Stores unique hashtags used in posts.

Key Attributes:

- id (Primary Key)
- name (Unique)

G. Table 7: post_hashtags

Implements a many-to-many relationship between posts and hashtags.

Key Attributes:

- post_id (Foreign Key → posts)
- hashtag_id (Foreign Key → hashtags)

H. Table 8: notifications

Stores notifications generated from user interactions.

Key Attributes:

- id (Primary Key)
- user_id (Receiver)
- actor_id (Action performer)
- type

I. Table 9: audit_logs

Maintains a record of system actions for tracking and auditing purposes.

Key Attributes:

- id (Primary Key)
- user_id
- action_type
- created_at

Note: No foreign key constraint is used to preserve logs even if users are deleted.

III. Relationships

Relationship	Type
User → Posts	1:N
User → Comments	1:N
Post → Comments	1:N
User ↔ Posts (Likes)	M:N
User ↔ User (Follows)	M:N (self-ref)
Post ↔ Hashtags	M:N
User → Notifications	1:N (×2)

Note: The User–Notifications relationship occurs twice: one where the user acts as the receiver of the notification, and another where the user acts as the initiator (actor) of the notification.

IV. ER DIAGRAM

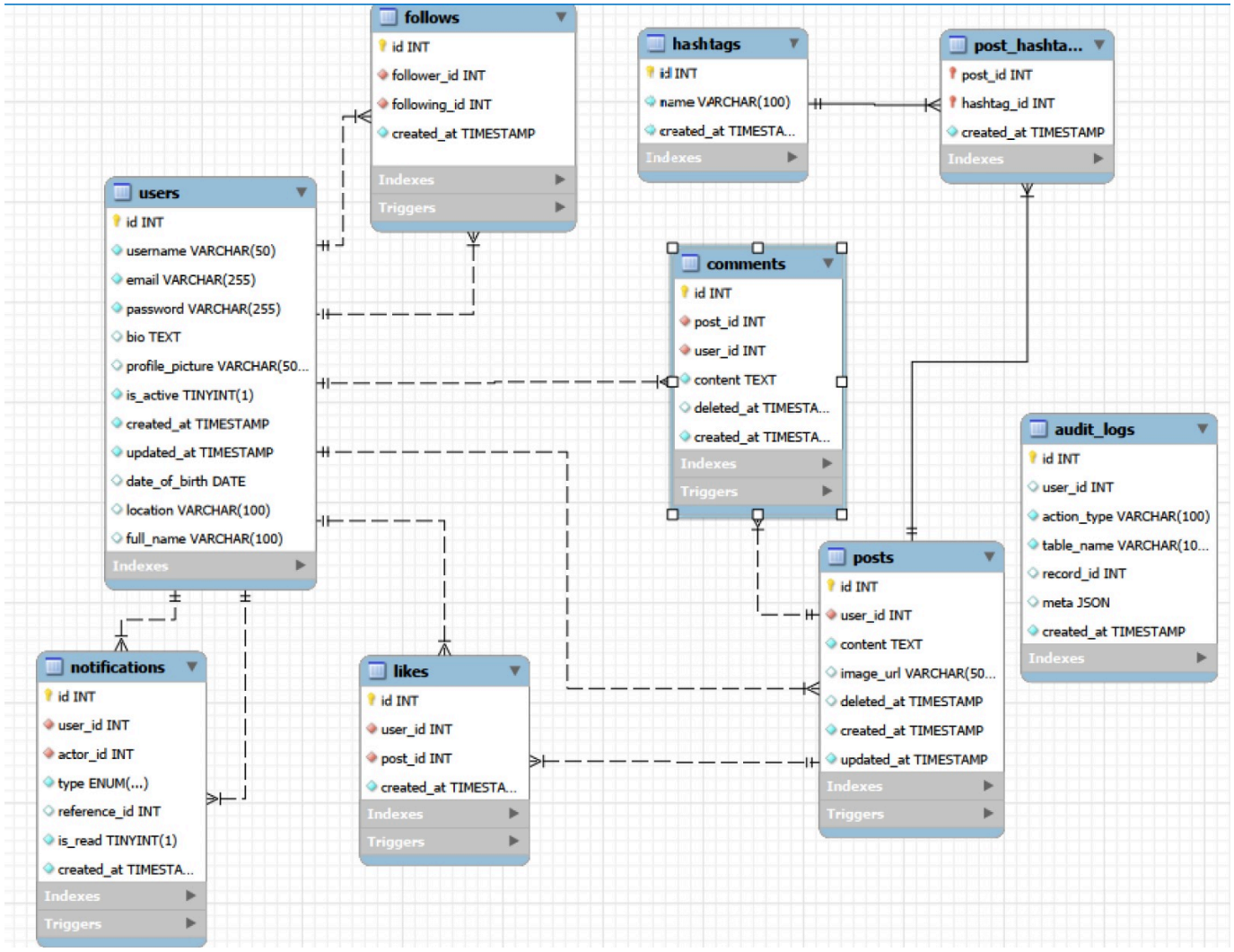

Figure 1: ER Diagram

The database consists of 9 tables organized around the core entities of a social media platform.

A. *Entity Relationships*

- A user can create multiple posts.
- A user can write multiple comments.
- A user can like multiple posts.
- A user can follow other users and can also be followed (self-referencing relationship).
- A user can receive notifications and also trigger notifications.
- A post can have multiple comments.
- A post can receive multiple likes.
- A post can be associated with multiple hashtags.
- A hashtag can be used in multiple posts (many-to-many relationship).
- A user can generate multiple audit log entries.

V. Implementation

A. Database Creation

This step involves creating the database used for the project and selecting it for further operations. The database serves as the foundation for storing all application data.

```
CREATE DATABASE IF NOT EXISTS storystream
CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

Fig. 1. Database creation and selection

B. Table Creation

In this stage, the core tables such as users and posts are created. These tables store essential data and define the basic structure of the system. Proper data types and primary keys are used to ensure efficient data storage.

```
CREATE TABLE IF NOT EXISTS users (
  id              INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  username        VARCHAR(50)   NOT NULL UNIQUE,
  email           VARCHAR(255)  NOT NULL UNIQUE,
  password        VARCHAR(255)  NOT NULL,
  bio             TEXT,
  profile_picture VARCHAR(500)  DEFAULT NULL,
  full_name       VARCHAR(100)  DEFAULT NULL,
  date_of_birth   DATE          DEFAULT NULL,
  location        VARCHAR(100)  DEFAULT NULL,
  is_active       TINYINT(1)    NOT NULL DEFAULT 1,
  created_at      TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at      TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,

  -- assertions to check constraints
  CONSTRAINT chk_username_min_len  CHECK (CHAR_LENGTH(username) >= 3),
  CONSTRAINT chk_email_format      CHECK (email LIKE '%_@__%.__%'),
  CONSTRAINT chk_password_hash_len CHECK (CHAR_LENGTH(password) >= 8),

  INDEX idx_users_username (username),
  INDEX idx_users_email    (email),
  INDEX idx_users_created_at (created_at)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

Fig. 2. Creation of users and posts tables

a) *Key Features Implemented:*

- **Constraints:** Primary key, foreign key, and unique constraints are applied to maintain data integrity and relationships.
- **Assertions:** Check constraints are used to validate conditions such as input length and logical correctness.

C. Triggers

Triggers are implemented to automate database operations and maintain an audit trail. For instance, a trigger is defined on the posts table that activates before an update operation. When a post is soft-deleted (i.e., the deleted_at field changes from NULL to a timestamp), the trigger automatically inserts a record into the audit_logs table. This ensures that all deletion activities are tracked without requiring manual intervention.

```
CREATE TRIGGER trg_audit_post_soft_delete
  BEFORE UPDATE ON posts
  FOR EACH ROW
BEGIN
  IF OLD.deleted_at IS NULL AND NEW.deleted_at IS NOT NULL THEN
    INSERT INTO audit_logs (user_id, action_type, table_name, record_id, meta)
    VALUES (
      NEW.user_id,
      'SOFT_DELETE_POST',
      'posts',
      NEW.id,
      JSON_OBJECT('trigger', 'trg_audit_post_soft_delete', 'deleted_at', NEW.deleted_at)
    );
  END IF;
END$$
DELIMITER ;
```

Fig. 3. Trigger implementation example

D. Sample Queries / Operations

Basic database operations such as inserting, retrieving, updating, and joining data are performed to demonstrate system functionality. These queries validate that the database is working correctly.

a) *INSERT:*

```
INSERT IGNORE INTO comments (post_id, user_id, content) VALUES
(1, 2, 'Congrats on the launch! What tech stack did you use? 🔥'),
(1, 3, 'Love the design! Super clean.'),
(2, 4, 'Totally agree — composite indexes changed my life.'),
(2, 5, 'Great tip! What was your query before and after?'),
(3, 1, 'K8s is love, K8s is life. Also check out Helm charts!'),
(7, 1, 'Technically accurate, but also beautiful when they work 😅'),
(8, 3, 'What indexes did you add specifically?'),
(8, 1, 'Index on (user_id, created_at) for feed queries works great.');
```

Fig. 4. INSERT into comments

b) *SELECT:*

```
const [postRows] = await pool.execute(
  'SELECT id FROM posts WHERE id = ? AND deleted_at IS NULL',
  [postId]
);
```

Fig. 5. SELECT implementation example

Note: Here, it checks whether a post exists in the database and is not deleted.

c) *SELECT with WHERE and ORDER BY:*

```
const [posts] = await pool.query(
  `SELECT
    p.id, p.content, p.image_url, p.created_at,
    u.id AS author_id, u.username, u.profile_picture,
    (SELECT COUNT(*) FROM likes    WHERE post_id = p.id)                      AS like_count,
    (SELECT COUNT(*) FROM comments WHERE post_id = p.id AND deleted_at IS NULL)  AS comment_count,
    EXISTS (SELECT 1 FROM likes WHERE user_id = ? AND post_id = p.id)         AS liked_by_me
  FROM posts p
  JOIN users u ON u.id = p.user_id
  WHERE p.deleted_at IS NULL
  ORDER BY p.created_at DESC
  LIMIT ? OFFSET ?`,
 [userId, limit, offset]
);
```

Fig. 6. SELECT implementation example

Note: This query is used to fetch a social media feed of posts. It retrieves each post's content, image, and creation time, along with the details of the user who created it such as username and profile picture by joining the posts and users tables. It also calculates how many likes and comments each post has, and checks whether the current user has liked the post or not. Only posts that are not deleted are included, and the results are sorted so that the newest posts appear first. Finally, LIMIT and OFFSET are used to display the posts in pages instead of showing all at once.

d) *JOIN:*

```
const [comments] = await pool.execute(
  //sql section
  `SELECT c.id, c.content, c.created_at,
          u.id AS author_id, u.username, u.profile_picture
   FROM comments c
   JOIN users u ON u.id = c.user_id
   WHERE c.id = ?`,

  [commentId]
);
```

Fig. 7. JOIN implementation example

Note: Here, it fetches a single comment from the database along with the user (author) details who wrote it.

e) *MULTI-JOIN:*

```
const [rows] = await pool.execute(
  `SELECT
    p.id, p.content, p.image_url, p.created_at,
    u.id AS author_id, u.username, u.profile_picture,
    (SELECT COUNT(*) FROM likes    WHERE post_id = p.id)                      AS like_count,
    (SELECT COUNT(*) FROM comments WHERE post_id = p.id AND deleted_at IS NULL) AS comment_count,
    EXISTS (SELECT 1 FROM likes WHERE user_id = ? AND post_id = p.id)         AS liked_by_me
  FROM posts p
  JOIN users u ON u.id = p.user_id
  WHERE p.id = ? AND p.deleted_at IS NULL`,
 [userId, postId]
);
```

Fig. 8. MULTI-JOIN implementation example

Note: This query fetches a single post by its ID along with the author's details by joining the users table. It also includes the total number of likes and comments on the post and checks whether the current user has liked it or not. Only posts that are not deleted are returned, and the result is a single detailed post object.

f) *DELETE:*

```
await pool.execute(
  'DELETE FROM likes WHERE user_id = ? AND post_id = ?',
  [userId, postId]
);
```

Fig. 9. DELETE implementation example

Note: Here, it removes a like from a post by a specific user.

g) *UPDATE:*

```
await pool.execute(
  'UPDATE comments SET deleted_at = CURRENT_TIMESTAMP WHERE id = ?',
  [commentId]
);
```

Fig. 10. UPDATE implementation example

Note: Here, it marks a comment as deleted in the database (soft delete).

h) *DATA CONTROL queries:*

```
CREATE USER 'storystream_app'@'%' IDENTIFIED BY 'secure_password';

GRANT SELECT, INSERT, UPDATE, DELETE ON storystream.* TO 'storystream_app'@'%';

REVOKE DROP, ALTER, CREATE ON storystream.* FROM 'storystream_app'@'%';
```

Fig. 11. GRANT and REVOKE implementation example

NOTE
1: CREATE USER: Creates a new database user storystream_app with the given password.
2: GRANT: Gives the user permission to read and modify data (SELECT, INSERT, UPDATE, DELETE) in the storystream database.
3: REVOKE: Removes administrative and structural privileges such as DROP, ALTER, and CREATE to prevent unauthorized modification of the database schema and ensure data integrity.

VI. TRANSACTION AND CONCURRENCY CONTROL

A. TRANSACTION USAGE

A transaction is a group of database operations that are executed as a single unit, meaning either all operations succeed together or all fail together. This ensures data consistency and prevents partial updates in case of errors or system failures. In MySQL, a transaction typically uses BEGIN, COMMIT, and ROLLBACK. When a transaction starts, changes are temporarily applied; if everything executes successfully, COMMIT permanently saves the changes to the database. However, if any error occurs, ROLLBACK undoes all changes made during the transaction, restoring the database to its previous state.

a) *beginTransaction: beginTransaction:*A database connection is first obtained from the connection pool, after which beginTransaction() is called to start the transaction

```
    await connection.beginTransaction();

```

Fig. 12. beginTransaction

b) *COMMIT:*

```
    await connection.commit();

```

Fig. 13. COMMIT

c) *ROLLBACK and RELEASE:*

```
  } catch (error) {
    await connection.rollback();
    next(error);
  } finally {
    connection.release();
  }
```

Fig. 14. ROLLBACK and RELEASE

COMMIT, ROLLBACK, RELEASE
Multiple INSERT or UPDATE operations are then executed within the transaction block. If all operations are successful, commit() is called to permanently save the changes to the database. If any error occurs during execution, rollback() is used to undo all changes made within the transaction, ensuring the database remains in a consistent state. Finally, release() is called to return the connection back to the pool for reuse, ensuring efficient resource management.

B. ACID Properties

The StoryStream database system follows all ACID properties to ensure reliable and consistent transaction processing.

Atomicity is maintained by wrapping every multi-table operation inside a BEGIN and COMMIT block. If any part of a transaction fails, a ROLLBACK is executed. For example, if hashtag linking fails during post creation, the entire post insertion is undone to maintain data integrity.

Consistency ensures that the database always moves from one valid state to another. This is enforced using constraints such as FOREIGN KEY, UNIQUE, and CHECK, along with triggers. For instance, a post cannot exist without a valid user, and a like cannot reference a deleted post.

Isolation is achieved using MySQL's InnoDB storage engine, which uses the default isolation level REPEATABLE READ to prevent issues such as dirty reads. Additionally, SELECT ... FOR UPDATE is used to apply pessimistic locking, preventing conflicts like lost updates during simultaneous like or follow operations.

Durability guarantees that once a transaction is committed, it will not be lost even in case of a system failure. This is ensured through InnoDB's write-ahead logging (WAL) mechanism and the configuration innodb_flush_log_at_trx_commit = 1, which forces immediate persistence of committed transactions.

C. CONCURRECY: ITS ISSUES AND SOLUTIONS

Concurrency in DBMS refers to the ability of the database to handle multiple users or operations at the same time without causing errors or inconsistent data. In a social media system like this project, many users may like posts, comment, or follow others simultaneously, so concurrency control ensures that all these operations are handled correctly. Here, concurrency is managed using transactions and database controls so that actions like likes, comments, and follows from multiple users are processed correctly without conflicts or data loss.

In this project, concurrency issues occur when multiple users perform actions at the same time, which can lead to inconsistent or incorrect data.

One major issue is the lost update (double-like race condition). This happens when two requests try to like the same post at the same time. Both requests first check if a like already exists, and since neither sees a record yet, both proceed to insert a like. This can result in duplicate or conflicting entries. To solve this, pessimistic locking is used with SELECT ... FOR UPDATE, which locks the row during the transaction. This ensures that one request completes first, while the other waits, preventing duplicate inserts.

```
const [existing] = await pool.execute(
  'SELECT id FROM likes WHERE user_id = ? AND post_id = ?',
  [userId, postId]
);
```

Fig. 15. SELECT ... FOR UPDATE

Another issue is the phantom read problem, which can occur in feed queries when a post is soft-deleted while another user is fetching the feed. To handle this, MySQL's REPEATABLE READ isolation level is used, which maintains a consistent snapshot of the data throughout the transaction. This ensures that the user sees stable and consistent results even when other concurrent changes are happening in the database.

D. Connection Pooling

Connection pooling is a technique used to manage and reuse database connections instead of creating a new connection every time a request is made. A group (pool) of connections is created in advance and reused whenever needed. This makes the system faster and more efficient.

In the StoryStream system, connection pooling is used to efficiently manage database connections. A maximum of 10 connections are allowed at the same time. If all connections are busy, new requests wait in a queue instead of failing. This helps prevent errors when many users access the system at once. The keepAlive option is used to keep connections active and avoid timeouts.

The pool automatically handles connections for us. When the system needs to run a query, it takes a connection using getConnection() or execute(). After the work is done, the connection is returned back to the pool using release(). This helps reuse connections, improves speed, and reduces load on the database.

```
const pool = mysql.createPool({
  host:            process.env.DB_HOST,
  user:            process.env.DB_USER,
  password:        process.env.DB_PASSWORD,
  database:        process.env.DB_NAME,
  port:            Number(process.env.DB_PORT),
  waitForConnections: true,
  connectionLimit:    10,
  queueLimit:         0,
  enableKeepAlive:    true,
  keepAliveInitialDelay: 0,
  ...sslConfig,
});
```

Fig. 16. Connection Pooling

VII. Assertion and Triggers

A. CHECK Constraints (Assertions)

StoryStream implements 9 CHECK constraints across 5 tables to enforce business rules at the database level.

1) chk_username_min_len (users)
CHAR_LENGTH(username) >= 3 ensures usernames have at least 3 characters.

2) chk_email_format (users)
email LIKE '%_@%.%' ensures a valid email structure.

3) chk_password_hash_len(users)
CHAR_LENGTH(password) >= 8 ensures password meets minimum length requirement.

4) chk_post_content_not_empty(posts)
CHAR_LENGTH(TRIM(content)) > 0 ensures post content is not empty.

5) chk_post_content_max_len(posts)
CHAR_LENGTH(content) <= 280 restricts posts to 280 characters.

6) chk_comment_content_not_empty(comments)
CHAR_LENGTH(TRIM(content)) > 0 ensures comments are not empty.

7) chk_comment_content_max_len(comments)
CHAR_LENGTH(content) <= 500 limits comment length to 500 characters.

8) chk_no_self_follow(follows)-
follower_id <> following_id prevents users from following themselves.

9) chk_no_self_notification (notifications)
user_id <> actor_id prevents self-generated notifications.

B. Database Triggers

Six triggers are defined in database/triggers.sql and loaded at startup by config/db.js:

a) *Trigger 1: trg_audit_post_soft_delete:*

```
CREATE TRIGGER trg_audit_post_soft_delete
  BEFORE UPDATE ON posts
  FOR EACH ROW
BEGIN
  IF OLD.deleted_at IS NULL AND NEW.deleted_at IS NOT NULL THEN
    INSERT INTO audit_logs (user_id, action_type, table_name, record_id, meta)
    VALUES (
      NEW.user_id,
      'SOFT_DELETE_POST',
      'posts',
      NEW.id,
      JSON_OBJECT('trigger', 'trg_audit_post_soft_delete', 'deleted_at', NEW.deleted_at)
    );
  END IF;
END$$
```

Fig. 17. Purpose: Automatically logs post soft-deletions to the audit trail, ensuring no deletion goes unrecorded even if the application code omits the audit insert.

b) *Trigger 2: trg_audit_comment_soft_delete:*

```
CREATE TRIGGER trg_audit_comment_soft_delete
  BEFORE UPDATE ON comments
  FOR EACH ROW
BEGIN
  IF OLD.deleted_at IS NULL AND NEW.deleted_at IS NOT NULL THEN
    INSERT INTO audit_logs (user_id, action_type, table_name, record_id, meta)
    VALUES (
      NEW.user_id,
      'SOFT_DELETE_COMMENT',
      'comments',
      NEW.id,
      JSON_OBJECT('trigger', 'trg_audit_comment_soft_delete', 'deleted_at', NEW.deleted_at)
    );
  END IF;
END$$
DELIMITER ;
```

Fig. 18. Purpose: Automatically logs post soft-deletions into the audit_logs table whenever deleted_at changes, ensuring all deletions are tracked even if not handled by the application.

c) *Trigger 3: trg_prevent_self_follow:*

```
CREATE TRIGGER trg_prevent_self_follow
  BEFORE INSERT ON follows
  FOR EACH ROW
BEGIN
  IF NEW.follower_id = NEW.following_id THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'A user cannot follow themselves.';
  END IF;
END$$
DELIMITER ;
```

Fig. 19. Purpose: prevents users from following themselves by blocking any insert where follower_id equals following_id and raising an error before the record is saved.

d) *Trigger 4: trg_prevent_self_notification:*

```
DELIMITER $$
CREATE TRIGGER trg_prevent_self_notification
  BEFORE INSERT ON notifications
  FOR EACH ROW
BEGIN
  IF NEW.user_id = NEW.actor_id THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Cannot create a notification where actor and receiver are the same user.';
  END IF;
END$$
DELIMITER ;
```

Fig. 20. Purpose: Prevents self-notifications by blocking inserts where user_id equals actor_id, ensuring a user cannot generate notifications for their own actions.

e) *Trigger 5: trg_cleanup_notifications_on_unlike:*

```
DELIMITER $$
CREATE TRIGGER trg_cleanup_notifications_on_unlike
  AFTER DELETE ON likes
  FOR EACH ROW
BEGIN
  DELETE FROM notifications
  WHERE actor_id      = OLD.user_id
    AND type          = 'like'
    AND reference_id = OLD.post_id;
END$$
DELIMITER ;
```

Fig. 21. Purpose: Automatically removes the "like" notification when a user unlikes a post by deleting the corresponding entry from the notifications table after a like is removed.

f) *Trigger 6: trg_cleanup_notifications_on_unfollow:*

```
CREATE TRIGGER trg_cleanup_notifications_on_unfollow
  AFTER DELETE ON follows
  FOR EACH ROW
BEGIN
  DELETE FROM notifications
  WHERE actor_id      = OLD.follower_id
    AND type          = 'follow'
    AND reference_id = OLD.follower_id
    AND user_id       = OLD.following_id;
END$$
DELIMITER ;
```

Fig. 22. Purpose: Automatically deletes follow notifications when a user unfollows someone, ensuring notification data stays consistent by removing related entries from the notifications table after a follow record is deleted.

VIII. SECURITY

A. Application-Level View Equivalents

While StoryStream does not define SQL VIEW objects in the schema, the controller layer implements the equivalent of restricted views through carefully crafted SELECT queries that limit data exposure:

a) *User-Profile View:* User-Profile View never exposes passwords. This query retrieves user profile information for display purposes and ensures that sensitive data such as passwords are never exposed. The password field is intentionally excluded from the SELECT statement, following security best practices and data minimization principles.

```
SELECT u.id, u.username, u.email, u.bio, u.profile_picture,
       u.full_name, u.date_of_birth, u.location, u.created_at
FROM users u WHERE u.username = ?;
```

Fig. 23. Simple SQL Query

```
const [rows] = await pool.execute(
  `SELECT
    u.id, u.username, u.email, u.bio, u.profile_picture,
    u.full_name, u.date_of_birth, u.location, u.created_at,
    (SELECT COUNT(*) FROM follows WHERE following_id = u.id) AS followers_count,
    (SELECT COUNT(*) FROM follows WHERE follower_id  = u.id) AS following_count,
    (SELECT COUNT(*) FROM posts    WHERE user_id = u.id AND deleted_at IS NULL) AS posts_count
   FROM users u
   WHERE u.username = ? AND u.is_active = 1`,
  [username]
);
```

Fig. 24. Node.js paginated query

b) *Notification View:* This query retrieves notifications for a user and enriches them with actor details by joining the users table. It helps display who performed the action (like, comment, follow) along with notification metadata.

```
SELECT n.id, n.type, n.reference_id, n.is_read, n.created_at,
       u.username AS actor_username, u.profile_picture AS actor_avatar
FROM notifications n
JOIN users u ON u.id = n.actor_id
WHERE n.user_id = ?;
```

Fig. 25. Simple SQL Query

```
const [notifications] = await pool.query(
  `SELECT
    n.id, n.type, n.reference_id, n.is_read, n.created_at,
    u.id AS actor_id, u.username AS actor_username, u.profile_picture AS actor_avatar
   FROM notifications n
   JOIN users u ON u.id = n.actor_id
   WHERE n.user_id = ?
   ORDER BY n.created_at DESC
   LIMIT ? OFFSET ?`,
  [userId, limit, offset]
);
```

Fig. 26. Node.js paginated query

IX. RESULT

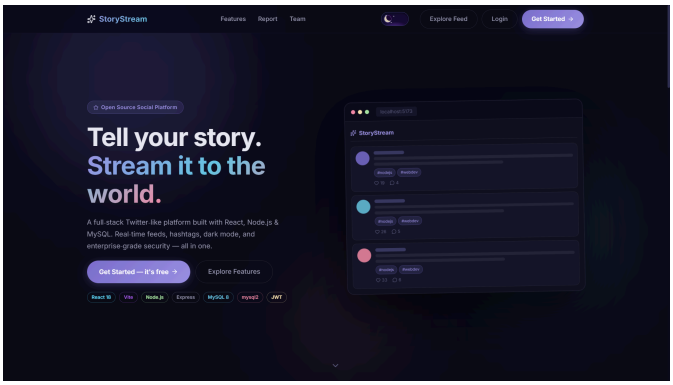

Fig. 27. Home Page

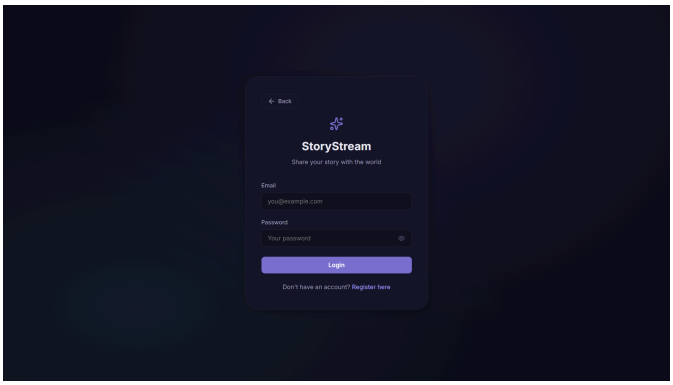

Fig. 28. Login Page

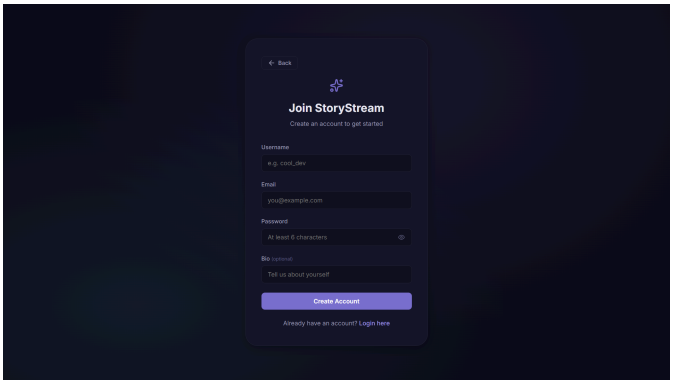

Fig. 29. Register Page

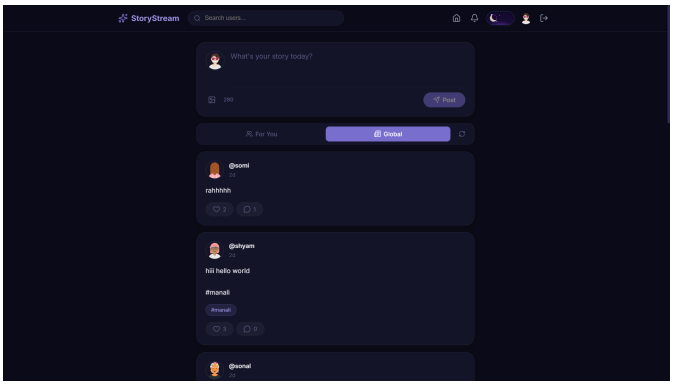

Fig. 30. Global Feed

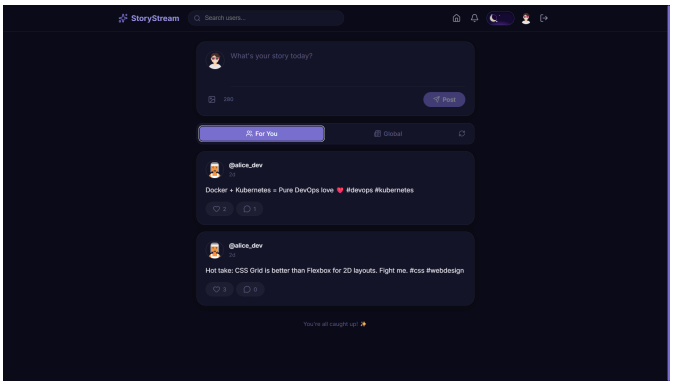

Fig. 31. For You Page

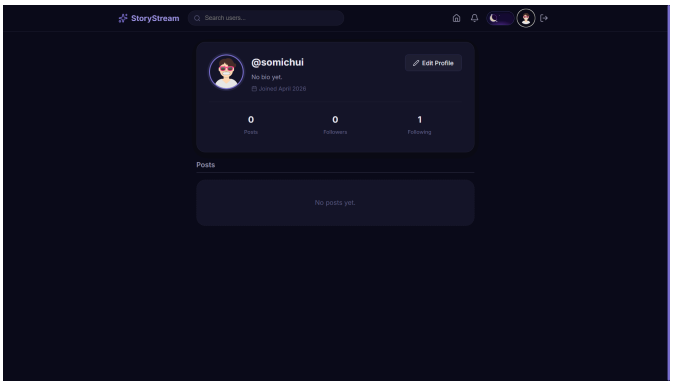

Fig. 32. User Page

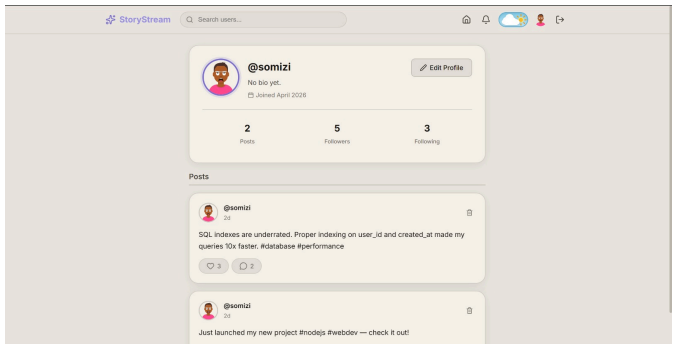

Fig. 33. Light Mode

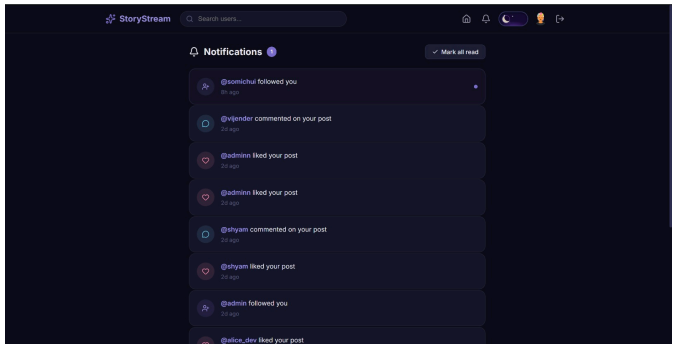

Fig. 34. Notifications Page

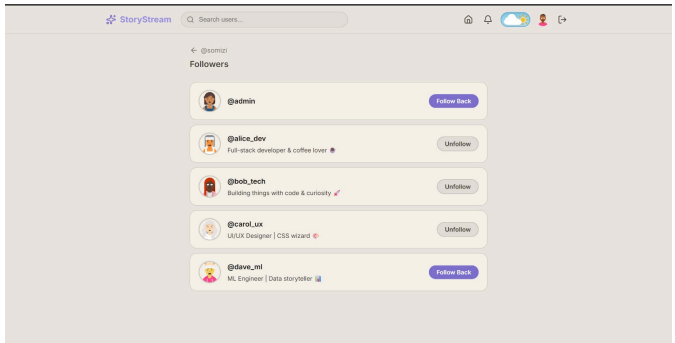

Fig. 35. Follower List Page

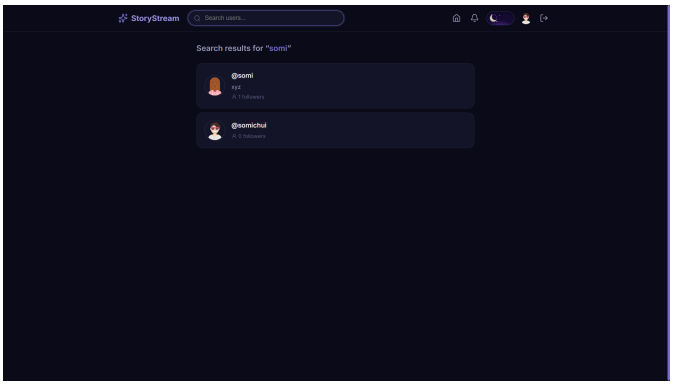

Fig. 36. Search Users

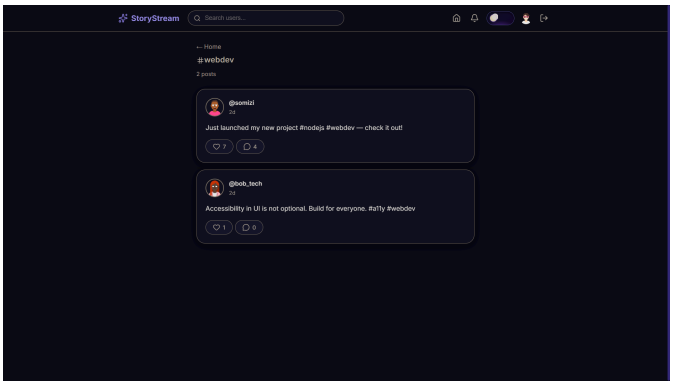

Fig. 37. Search by HASHTAGS

X. CONCLUSION

The StoryStream database project successfully demonstrates the design and implementation of a scalable, relational backbone for a modern social media platform. By transitioning from conceptual requirements to a fully realized MySQL database, the project achieved several key milestones in database engineering:

1) Architectural Integrity

The system was designed with 3NF/BCNF normalization at its core, ensuring data redundancy is minimized across the 9 primary tables. From the core users and posts entities to complex many-to-many structures like post_hashtags, the schema maintains a clean separation of concerns.

2) Relational Robustness

Through the use of the ER Diagram, clear relationships were established, such as the self-referencing follows table and the polymorphic-style notifications system. The use of the InnoDB engine was pivotal, providing the necessary support for Foreign Key constraints and ensuring ACID compliance for every transaction, from liking a post to registering a new user.

3) Advanced Database Logic

Implementation went beyond simple storage by utilizing MySQL Triggers and Check Constraints. These provide "defense-in-depth," automating audit logs for soft deletes and preventing illegal states—such as users following themselves—directly at the database level.

4) Implementation Success

The project successfully integrated modern development tools, including Node.js and the Prisma ORM, to bridge the gap between application logic and the database. By implementing a "Least Privilege" security model with the storystream_app user, the system is not only functional but also architecturally secure against unauthorized schema modifications.

5) Final Summary

In summary, StoryStream stands as a high-performance relational database capable of handling the complex interdependencies of social media data. The final implementation provides a fast, consistent, and audit-ready environment that fulfills all the requirements of a production-ready data layer.

XI. FUTURE SCOPE

While the current version of StoryStream provides a robust relational foundation, several enhancements can be integrated to transform it into a production-ready social media ecosystem:

Real-time Communication: The platform can be enhanced with WebSockets (Socket.io) to enable real-time features such as instant messaging, live notifications, and activity updates without requiring page refresh, improving user engagement.

Personalized Feed System: A recommendation engine can be introduced to analyze user behavior, interactions, and hashtag usage to generate a personalized "For You" feed, improving content relevance.

Media Storage & Optimization: Integration with cloud storage services like AWS S3 or Cloudinary can be used to efficiently handle images and videos, along with CDN support for faster content delivery and reduced server load.

Caching for Performance: Implementing Redis or similar caching systems can help store frequently accessed data such as trending posts, user sessions, and notifications, significantly improving response time and scalability.

Scalability & Deployment: The application can be containerized using Docker and orchestrated with Kubernetes, enabling smooth deployment on cloud platforms like AWS or GCP and supporting horizontal scaling under heavy traffic.

Enhanced Security: Security can be strengthened by adding Multi-Factor Authentication (MFA), OAuth 2.0 login integration (Google/GitHub), and improved token management to ensure safer user authentication and authorization.

XII. References

1) Node.js Documentation — Available at: https://nodejs.org
2) Express.js Guide — Available at: https://expressjs.com
3) MySQL 8.0 Reference Manual — Available at: https://dev.mysql.com/doc/
4) MDN Web Docs — Available at: https://developer.mozilla.org
5) GeeksforGeeks – DBMS and SQL Tutorials — https://www.geeksforgeeks.org
6) Elmasri, Navathe – Fundamentals of Database Systems (Book) — https://asolanki.co.in/wp-content/uploads/2019/02/Fundamentals_of_Database_Systems_6th_Edition-1.pdf